

Lia — Phase Roadmap

What's done, what's next, and what's deliberately not scheduled.

What's actually done, what's next, and what's deliberately not scheduled — kept here so a decision doesn't have to be re-derived from memory or git log next time. Rebuilt whenever a phase finishes; see the Growth Log for the narrative version of how she got here.

Phase 1 — Polish what's built · done

Three small, independent items, chosen over widening her actions or chasing raw intelligence first. All three landed.

1.1 — Instrument the cloud fallback

So the free/paid/local model choice gets decided from a week of real data instead of one session's impression. Every rate-limit and every mid-stream drop now writes a timestamped line to `cloud_fallback_log.jsonl`, and `fallback_report.py` reads it back as a plain count.

Already earning its keep: one real fallback landed within days of shipping this — a rate limit on the free conversation model, announced and absorbed rather than left silent.

1.2 — Voice-ID status, spoken

`/whoami` was console-only, unreachable from the tray app most actual use happens in. “Do you know my voice”, “is that me” and similar now reach the same status out loud.

1.3 — Forget a single note

There was no way to remove one stored note short of touching the database directly. `db.delete_fact()` plus “what have you remembered” / “forget number 2” — numbered list, pick by position, the same pattern already proven for music.

Asking someone else · done, not originally scoped

Added mid-stream, at request, rather than planned in the phase above: “ask cat for a suggestion” reaches a second, named free model and relays the answer attributed. Two names today — cat, fox — each measured and picked for being fast *and* reliable, not for novelty.

Not the multi-agent idea this plan shelved below. That idea split *her own* reasoning across several small local models. This is the opposite shape: one cloud call, on request, to a model she isn't and doesn't pretend to be, at zero local VRAM cost. Closer to the “look that up” internet path than to anything rejected.

Sixteen free OpenRouter models were catalogued directly and eight measured for real, twice. The metric mattered: this path collects the whole reply before speaking any of it, so total time is what a person waits on, not time to first word — the first pick (measured on the wrong metric) was heavier than it needed to be, and swapped once that was noticed. Two further names were tried and dropped rather than kept as a weak link: one lightest-on-paper replacement worked once and then failed empty five times in a row on a re-test; another was fast when it worked but wrong close to a third of the time across repeated real testing, each failure a real wait rather than an answer. Two names that have never once failed beat a bigger roster with a weak link in it.

A real bug surfaced along the way, unrelated to model choice: a streamed request's timeout only bounds the gap between chunks, not the whole call, so a slow provider could hang far past it. Measured directly at 121 seconds, still empty. Every ask is now capped on wall-clock time regardless of provider behavior.

A typed panel and a new icon · done, not originally scoped

The tray app's first window. A small typed panel now opens alongside the tray icon, styled after Claude Code's own terminal — white and orange. Built as a second interface to the exact same conversation pipeline the console already had, not a new one: it satisfies the same typed-input and printed-output shapes `main.py` already knew how to use, so nothing about the conversation loop itself needed to change.

`LISTEN_ENABLED` is off while this is the current way of working with her — a config flag, not commented-out code, specifically so the tray's own **Listening** checkbox keeps working live, no restart, the moment voice is wanted again.

The tray icon changed too: a purple cat face, replacing the plain warm dot. Same two states (filled/listening, hollow/not) kept rather than reinvented; the mouth is the one new signal — open for speaking, closed for idle — doing the same job the old inner dot did, more legibly.

The one real risk was threading, and it was checked, not assumed. Tkinter and the tray icon's own loop both want the thread that started them. Verified before writing any of it rather than after something froze: the tray library documents running off the main thread as unsafe in general, safe specifically on Windows — which is what this project has been throughout, so that's the trade taken.

Phase 2 — A background summarizer · next

Of the three “raise the ceiling” ideas that came out of the intelligence conversation, this is the one with no architectural friction.

CLOUD_HISTORY_TURNS is a hard cutoff — anything older than four turns just falls off the edge of what she sends. A summarizer removes that cliff cheaply because it reuses a pattern that already exists twice: the diary and fact-extraction both run *off the hot path*, once, locally, at session rollover — not on every turn.

- A new SUMMARY_PROMPT in config.py, same shape as DIARY_PROMPT.
- A session_summaries table in db.py.
- A call from end_session(), alongside the diary and fact-extraction calls already there.
- A build_summary_context() injector — doesn't exist yet, since the diary has no equivalent — that folds the last summary into what CLOUD_HISTORY_TURNS sends.

Zero added latency on a live turn, as long as it stays scoped to session rollover the same way the diary already is.

How to know it worked

Hold a session past four turns, end it, confirm a new row in session_summaries. Start a fresh session and check — via CLOUD_LOG_USAGE's token count, or a debug print — that the summary is actually part of what gets sent, not just written and ignored.

Phase 3 — Grounding check, log-only · optional, lower priority

The other “raise the ceiling” idea, with a real conflict worth knowing before starting: replies stream to speech sentence-by-sentence on purpose — speaking sentence one while the rest still generates is exactly the latency-hiding win already measured and built. A verifier that blocks speech until it checks the whole reply undoes that directly. A per-sentence check has its own problem: a lone sentence like “yes, exactly” can't be judged alone, and it's a call per sentence, not per turn.

So: log-only first, if at all. Check the completed reply against whatever memory or library context was already retrieved for that turn — no new retrieval needed, it's already materialized before the cloud call — *after* she's already spoken, and log a mismatch rather than gate on it. That's a real signal on how often the cloud model actually invents something, without touching the sentence streaming that already works. A blocking version, if it ever happens, is a separate decision made from what the log-only pass finds.

How to know it worked

Trigger a reply that draws on retrieved context, confirm a log line records the check. Separately: time-to-first-word before and after, to confirm it's unchanged — this phase's entire point is adding the check without paying the latency its own design is built to avoid.

Explicitly not doing right now

Why not, for now

Only reachable through AUTO_EXTRACT_FACTS, which stays off by choice. Revisit only if that changes.

No spare VRAM for stacked local calls on a 4GB card already near capacity. The role split it would add already decomposed query that collapses to yes/no is just another judge.py. (Distinct from “Asking someone else” ab

Not a current priority. Two real prerequisites whenever it is picked up: intent.py's tools are deliberately param voice-ID is the natural consent gate for it but is unenrolled and unvalidated against a real human voice today.

Single-machine, Windows-specific project today. Not raised as a priority.

Open questions

- **Which model carries the actual conversation.** Local (2.19s, free, least capable), paid gpt-4o-mini (2.51s, ~\$0.0001/turn), or the current free model (4.53s, rate-limited but no cost) — still an open call, not a settled one, revisit as real fallback data accumulates via 1.1 above.
- **A third (or fourth) “ask” name.** Two were tried and dropped rather than kept as weak links. Worth revisiting if OpenRouter's free catalog turns up something both light and consistently reliable — re-measure on total time, the way cat and fox were, before adding one back.